
Pick & Serve

Predictor de Partidos ATP

White Paper Técnico

Resumen Ejecutivo

Pick & Serve es una plataforma web de predicción de partidos de tenis ATP, construida sobre una arquitectura orientada a eventos con RabbitMQ como broker de mensajería. El sistema permite a múltiples usuarios competir en un ranking en tiempo real, realizando pronósticos sobre partidos de torneos ATP. El presente documento describe la arquitectura técnica, decisiones de diseño, stack tecnológico y flujos de datos del sistema.

Materia: Ingeniería de Software II
Tipo: Trabajo Práctico — Prueba de Concepto
Tecnologías: FastAPI · React · RabbitMQ · PostgreSQL
Versión: 1.0.0
Fecha: Junio 2026

Índice

1. Introducción y Contexto	2
1.1. Descripción del Problema	2
1.2. Propuesta de Valor	2
1.3. Alcance del Sistema	2
2. Arquitectura General del Sistema	2
2.1. Visión de Alto Nivel	2
2.2. Diagrama de Componentes	3
2.3. Principios de Diseño Aplicados	3
3. Arquitectura Orientada a Eventos	3
3.1. Motivación para EDA	3
3.2. Exchange y Routing Keys	4
3.3. Flujo Completo: Carga de Resultado	4
3.4. Encadenamiento de Eventos (Event Chaining)	4
4. Stack Tecnológico	5
4.1. Backend — FastAPI	5
4.2. Frontend — React + TypeScript	5
4.2.1. Diseño Visual — Sistema de Tokens	5
4.3. Broker de Mensajería — RabbitMQ	6
4.4. Base de Datos — PostgreSQL	6
4.5. Infraestructura — Docker Compose	6
5. Modelo de Datos	7
5.1. Entidades Principales	7
5.1.1. Predicciones y Scoring	7
5.1.2. Estado de Partidos y Jornadas	7
5.2. Flujo de Estados	7
6. Workers Asíncronos	7
6.1. Arquitectura de Workers	7
6.2. Scoring Worker	8
6.3. Notification Worker	8
6.4. Ranking Worker	8
6.5. Scheduler	8
7. Interfaz de Usuario	8
7.1. Diseño Responsive	8
7.2. Vistas del Sistema	9
7.2.1. Login	9
7.2.2. Dashboard de Pronósticos	9
7.2.3. Rankings	9
7.2.4. Estadísticas Personales	9
7.2.5. Notificaciones	9
7.2.6. Panel de Administración	10

8. Patrones y Decisiones de Diseño	10
8.1. Patrón Publisher/Subscriber	10
8.2. Patrón CQRS Ligero	10
8.3. Idempotencia en Workers	10
8.4. Polling vs WebSocket	10
9. Deployment e Infraestructura	10
9.1. Contenedores Docker	11
9.2. Configuración Nginx	11
9.3. CI/CD	11
10. Análisis de Calidad	11
10.1. Puntos Fuertes	11
10.2. Limitaciones y Trabajo Futuro	12
11. Conclusiones	12

1. Introducción y Contexto

1.1. Descripción del Problema

Los sistemas de predicción deportiva presentan desafíos técnicos particulares que los hacen idóneos para estudiar arquitecturas de software modernas:

- **Alta concurrencia:** múltiples usuarios enviando predicciones simultáneamente.
- **Consistencia de datos:** los puntajes deben calcularse correctamente aunque varios eventos ocurran en paralelo.
- **Actualización en tiempo cuasi-real:** el ranking debe reflejar los resultados apenas se cargan los datos, sin requerir recarga manual.
- **Desacoplamiento:** las acciones administrativas (cargar resultados, cerrar jornadas) no deben bloquear al usuario final.

1.2. Propuesta de Valor

Pick & Serve aborda estos desafíos implementando una **arquitectura orientada a eventos (Event-Driven Architecture, EDA)** donde cada acción relevante del sistema se traduce en un evento publicado en un broker de mensajería, permitiendo que múltiples consumidores reaccionen de forma independiente y asíncrona.

1.3. Alcance del Sistema

El sistema cubre el flujo completo:

1. Autenticación de jugadores y administradores.
2. Carga de pronósticos por parte de los jugadores sobre partidos abiertos.
3. Carga de resultados por parte del administrador.
4. Puntuación automática via workers asíncronos.
5. Notificaciones a jugadores sobre resultados y cierres de jornada.
6. Ranking global actualizado automáticamente.

2. Arquitectura General del Sistema

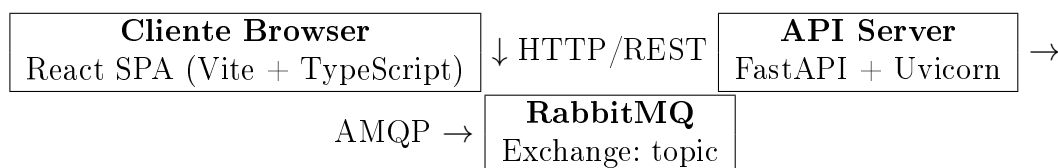
2.1. Visión de Alto Nivel

El sistema se organiza en cinco capas principales que se comunican de forma desacoplada:

Capas del Sistema

1. **Frontend (React + TypeScript)** — Interfaz web responsive (SPA).
2. **Backend API (FastAPI)** — Endpoints REST, lógica de negocio, publicación de eventos.
3. **Broker de Mensajería (RabbitMQ)** — Exchange topic para routing de eventos.
4. **Workers Asíncronos (Python)** — Consumidores especializados: scoring, notificaciones, ranking.
5. **Persistencia (PostgreSQL)** — Base de datos relacional con SQLAlchemy ORM.

2.2. Diagrama de Componentes



RabbitMQ distribuye eventos a tres colas paralelas: `scoring.queue` · `notifications.queue` · `ranking.queue`

2.3. Principios de Diseño Aplicados

Separación de responsabilidades (SoC) Cada worker tiene una única responsabilidad bien definida.

Alta cohesión / Bajo acoplamiento Los módulos interactúan únicamente via contratos de eventos (mensajes JSON).

Idempotencia Los workers pueden procesar el mismo mensaje múltiples veces sin generar efectos adversos.

Resiliencia El broker persiste mensajes; si un worker cae, los mensajes se encolan y procesan al recuperarse.

3. Arquitectura Orientada a Eventos

3.1. Motivación para EDA

La arquitectura orientada a eventos fue seleccionada por varias razones fundamentales:

¿Por qué EDA?

En un sistema donde un solo resultado de partido impacta en: puntajes de N usuarios, generación de N notificaciones, y recalcule del ranking global — procesar todo de forma síncrona generaría una latencia de respuesta propor-

cional a N. Con EDA, el administrador recibe confirmación inmediata y el procesamiento ocurre en paralelo en background.

3.2. Exchange y Routing Keys

Se utiliza un **exchange de tipo topic** en RabbitMQ, que permite routing flexible basado en patrones:

Evento	Routing Key	Productores
Resultado cargado	<code>match.result.loaded</code>	API (Admin endpoint)
Scores actualizados	<code>scores.updated</code>	Scoring Worker (encadenamiento)
Jornada cerrada	<code>round.closed</code>	API (Admin endpoint)

3.3. Flujo Completo: Carga de Resultado

1. El administrador POST a `/api/admin/matches/{id}/result`.
2. El backend persiste el resultado en PostgreSQL y publica:

```

1 await publisher.publish(
2   routing_key="match.result.loaded",
3   body={
4     "match_id": match.id,
5     "winner_player_id": winner,
6     "tournament_id": match.tournament_id,
7   }
8 )

```

Listing 1: Publicación del evento

3. RabbitMQ enruta el mensaje a **tres colas en paralelo** (fan-out mediante bindings):
 - **Scoring Worker:** calcula puntos para cada predicción del partido. Publica `scores.updated` al finalizar.
 - **Notification Worker:** genera notificación para cada jugador que predijo.
 - **(inicio del chain)** El Scoring Worker encadena hacia Ranking.
4. El **Ranking Worker** consume `scores.updated` y recalcula posiciones globales.

3.4. Encadenamiento de Eventos (Event Chaining)

Un patrón clave en el diseño es el encadenamiento: el Scoring Worker no solo actualiza puntajes, sino que al completar su trabajo publica un nuevo evento `scores.updated`. Esto permite que el Ranking Worker sea un consumidor desacoplado que no necesita saber nada de partidos o predicciones — solo reacciona cuando los scores cambian.

Este patrón es análogo al patrón **Saga** en microservicios.

4. Stack Tecnológico

4.1. Backend — FastAPI

Tecnología	Justificación
FastAPI 0.111	Framework async nativo en Python, tipado via Pydantic, documentación OpenAPI automática.
SQLAlchemy 2.0	ORM moderno con soporte async, migrations, y type-safe queries.
Pydantic 2.7	Validación de datos en entrada/salida, serialización automática.
Uvicorn	Servidor ASGI de alto rendimiento basado en uvloop.
aio-pika 9.4	Cliente RabbitMQ async para Python, soporta reconexión automática.

La API expone 6 routers RESTful:

- `/api/auth` — Login y listado de usuarios
- `/api/rounds` — Jornadas y partidos
- `/api/predictions` — CRUD de pronósticos
- `/api/admin` — Carga de resultados, cierre de jornadas
- `/api/ranking` — Tabla de posiciones global
- `/api/notifications` — Alertas por usuario

4.2. Frontend — React + TypeScript

Tecnología	Justificación
React 18.3	Biblioteca UI con virtual DOM, hooks, componentes reutilizables.
TypeScript 5.4	Tipado estático end-to-end, mejor DX y refactoring seguro.
Vite 5.2	Build tool ultra-rápido con HMR, optimización de bundles.
React Router 6	Routing client-side declarativo.
Tailwind CSS 3	Utility-first CSS con design tokens personalizados.
Material Symbols	Iconografía consistente con sistema de diseño Google Material 3.

4.2.1. Diseño Visual — Sistema de Tokens

El frontend implementa un sistema de diseño oscuro inspirado en Material You (Material Design 3) con los siguientes tokens de color primarios:

Token	Valor Hex	Uso
surface	#101415	Fondo principal
surface-container	#1d2022	Cards y paneles
orange-accent	#FF5C00	CTAs, highlights, énfasis
secondary-container	#ff5e07	Nav activo, botones primarios
on-surface	#e0e3e5	Texto principal
outline-variant	#44474e	Bordes y separadores

4.3. Broker de Mensajería — RabbitMQ

RabbitMQ fue seleccionado sobre alternativas como Kafka o Redis Pub/Sub por:

- **Persistencia de mensajes:** garantía de entrega incluso con workers caídos.
- **Exchange tipo topic:** permite routing flexible con routing keys jerárquicas.
- **Management UI:** interfaz visual para monitoreo de colas y mensajes.
- **Mature ecosystem:** cliente `aio-pika` con soporte async nativo.
- **Reconexión automática:** el patrón base en `workers/base.py` maneja desconexiones transparentemente.

4.4. Base de Datos — PostgreSQL

Tabla	Descripción
users	Jugadores y administradores
tournaments	Torneos ATP (ej: US Open, Wimbledon)
rounds	Jornadas de predicción por torneo
matches	Partidos individuales, con flag <code>is_final</code>
predictions	Pronósticos usuario-partido con puntos calculados
rankings	Posición y puntos acumulados por usuario
notifications	Alertas por usuario con estado de lectura

4.5. Infraestructura — Docker Compose

El sistema completo se orquesta con Docker Compose en 5 servicios:

```

1 services:
2   frontend    # React/Nginx - puerto 80
3   backend     # FastAPI - puerto 8000
4   workers     # Python workers (scoring + notif + ranking +
               scheduler)
5   postgres    # PostgreSQL - puerto 5432
6   rabbitmq    # RabbitMQ - AMQP 5672, Management UI 15672

```

Listing 2: Servicios Docker Compose

5. Modelo de Datos

5.1. Entidades Principales

5.1.1. Predicciones y Scoring

El modelo de puntuación es simple y auditable:

Regla de Puntuación

- **Pronóstico correcto:** +10 puntos base.
- **Partido marcado como FINAL** (`is_final = true`): +2 puntos adicionales.
- **Pronóstico incorrecto:** 0 puntos (sin penalización).
- El campo `predictions.points` es `NULL` hasta que el partido se resuelve.

5.1.2. Estado de Partidos y Jornadas

Entidad	Estados posibles
<code>Round.status</code>	<code>open</code> → <code>closed</code>
<code>Match.status</code>	<code>pending</code> → <code>finished</code>

Los usuarios solo pueden cargar o modificar pronósticos cuando la jornada está `open` y el partido está `pending`. Este invariante se valida en el backend.

5.2. Flujo de Estados

Jornada: `open` $\xrightarrow{\text{Admin cierra}}$ `closed`

Partido: `pending` $\xrightarrow{\text{Admin carga resultado}}$ `finished`

Predicción: `points = NULL` $\xrightarrow{\text{Scoring worker}}$ `points = N`

6. Workers Asíncronos

6.1. Arquitectura de Workers

Todos los workers heredan de una clase base `BaseWorker` definida en `workers/base.py`, que encapsula:

- Conexión y reconexión automática a RabbitMQ via `aio-pika`.
- Configuración del exchange y bindings.
- Manejo de errores y logging.

6.2. Scoring Worker

Consume: `match.result.loaded`

Responsabilidades:

1. Obtiene todas las predicciones del partido resuelto.
2. Para cada predicción, calcula los puntos según las reglas de negocio.
3. Actualiza `predictions.points` en PostgreSQL.
4. Publica `scores.updated` para triggear el recálculo de ranking.

6.3. Notification Worker

Consume: `match.result.loaded`, `round.closed`

Responsabilidades:

1. Por `match.result.loaded`: genera una notificación por cada usuario que predijo el partido, indicando si acertó o no.
2. Por `round.closed`: genera una notificación global a todos los jugadores activos indicando el cierre de la jornada.

6.4. Ranking Worker

Consume: `scores.updated`

Responsabilidades:

1. Agrega puntos totales por usuario desde la tabla `predictions`.
2. Calcula conteo de aciertos y total de predicciones evaluadas.
3. Actualiza (UPSERT) la tabla `rankings` con posiciones recalculadas.
4. Las posiciones se asignan por `RANK() OVER (ORDER BY total_points DESC)`.

6.5. Scheduler

El sistema incluye un **scheduler** (`workers/scheduler.py`) que monitorea automáticamente el estado de las jornadas y puede cerrarlas cuando vence su deadline, publicando el evento `round.closed` sin intervención manual del administrador.

7. Interfaz de Usuario

7.1. Diseño Responsive

El frontend implementa un layout adaptativo con dos modos:

Mobile (<1024px) Top bar de 56px con logo + usuario, contenido en full-width, bottom nav redondeada de 70px con 4 tabs (Dashboard, Rankings, Alertas, Stats).

Desktop (≥ 1024 px) Sidebar fija de 256px a la izquierda con navegación vertical, contenido principal con padding lateral de 32px.

7.2. Vistas del Sistema

7.2.1. Login

Selección de cuenta simplificada — múltiples usuarios pre-seeded para demostración. Identifica automáticamente cuentas de administrador.

7.2.2. Dashboard de Pronósticos

Vista principal del jugador. Muestra:

- Barra de progreso de pronósticos completados en la jornada activa.
- Cards de partidos con selector radio para elegir ganador.
- Indicadores visuales de partidos bloqueados, finalizados y puntos obtenidos.

7.2.3. Rankings

- Podio visual para top 3 (oro, plata, bronce) con gradientes de color.
- Lista completa con posición, iniciales, aciertos y puntos.
- Auto-actualización cada 5 segundos (polling).
- Indicador de live con timestamp de última actualización.

7.2.4. Estadísticas Personales

- Ring chart SVG de precisión general.
- Grid de 4 métricas: puntos totales, posición global, aciertos, evaluados.
- Historial completo de pronósticos con estado (pendiente/correcto/incorrecto).

7.2.5. Notificaciones

- Lista cronológica de alertas por tipo (resultado, cierre de jornada).
- Iconografía codificada por color: verde (acierto), rojo (cierre), naranja (info).
- Marcado de leído al hacer clic. Badge de no leídos en el nav.

7.2.6. Panel de Administración

- Métricas: jornadas abiertas y total de partidos pendientes.
- Tarjeta explicativa de la arquitectura de eventos (transparencia técnica).
- Por cada jornada: botón de cierre y cards por partido con botones de resultado.
- Al cargar resultado: dispara el pipeline de eventos completo.

8. Patrones y Decisiones de Diseño

8.1. Patrón Publisher/Subscriber

El sistema implementa Pub/Sub clásico donde:

- El **publisher** (API) no sabe quiénes son los subscribers.
- Los **subscribers** (workers) no saben quién publicó el evento.
- El **broker** (RabbitMQ) actúa como intermediario desacoplado.

8.2. Patrón CQRS Ligero

Si bien no es CQRS estricto, el sistema separa semánticamente:

- **Commands**: acciones que modifican estado (POST/PUT endpoints, events).
- **Queries**: lecturas de solo lectura (GET /ranking, GET /predictions).

8.3. Idempotencia en Workers

Los workers están diseñados para ser idempotentes: procesar el mismo evento dos veces produce el mismo resultado que procesarlo una vez. Esto es crítico para garantizar consistencia ante fallos y reenvíos del broker.

8.4. Polling vs WebSocket

La actualización del ranking utiliza polling cada 5 segundos desde el frontend. Esta decisión fue deliberada:

- **Simplicidad**: no requiere infraestructura de WebSocket.
- **Suficiencia**: el ranking no requiere actualización sub-segundo.
- **Correctness**: cada fetch garantiza datos frescos de PostgreSQL.

9. Deployment e Infraestructura

9.1. Contenedores Docker

Cada servicio tiene su propio `Dockerfile` optimizado:

- **Frontend:** Multi-stage build — Node para compilar, Nginx Alpine para servir. El bundle resultante es de ~196KB (60KB gzip).
- **Backend:** Python 3.x con dependencias en `requirements.txt`.
- **Workers:** Imagen compartida base con el backend, punto de entrada diferente.
- **PostgreSQL:** Imagen oficial con volumen persistente.
- **RabbitMQ:** Imagen con management plugin habilitado.

9.2. Configuración Nginx

El frontend incluye configuración Nginx (`nginx.conf`) con:

- Serving de archivos estáticos con cache headers.
- Fallback a `index.html` para client-side routing (SPA mode).
- Gzip compression habilitada.

9.3. CI/CD

El repositorio incluye GitHub Actions configurados en `.github/` para automatizar el pipeline de integración y deployment continuo.

10. Análisis de Calidad

10.1. Puntos Fuertes

1. **Escalabilidad horizontal:** los workers son stateless y se pueden escalar simplemente aumentando instancias — RabbitMQ distribuye la carga automáticamente.
2. **Resiliencia ante fallos:** si un worker cae, los mensajes permanecen en la cola y se procesan al reiniciar. No hay pérdida de datos.
3. **Observabilidad:** la Management UI de RabbitMQ permite ver en tiempo real el estado de colas, mensajes pendientes y throughput.
4. **Type safety end-to-end:** TypeScript en frontend + Pydantic en backend minimizan errores en tiempo de ejecución.
5. **Developer Experience:** Vite HMR para desarrollo frontend, FastAPI auto-docs en `/docs` para testing de API.

10.2. Limitaciones y Trabajo Futuro

Limitación	Mejora Propuesta
Polling de ranking (5s)	Migrar a WebSocket o SSE para updates push.
Auth por selección de user	Implementar JWT con contraseñas hasheadas (bcrypt).
Sin tests automatizados	Agregar pytest para backend, Vitest para frontend.
Workers en monolito	Separar en microservicios independientes con su propio repositorio.
Sin rate limiting	Implementar límites por IP/usuario en la API.

11. Conclusiones

Pick & Serve demuestra en forma concreta cómo una **arquitectura orientada a eventos** resuelve el problema del acoplamiento temporal en sistemas con múltiples efectos secundarios.

La elección de RabbitMQ como broker permitió implementar un fan-out genuino — un único evento de resultado de partido dispara concurrentemente scoring, notificaciones y (vía encadenamiento) actualización de ranking — sin que ninguno de estos procesos conozca la existencia de los demás.

El stack tecnológico (FastAPI + React + PostgreSQL) representa una elección pragmática que prioriza productividad sin sacrificar performance, y es representativo de lo que se usa en la industria para sistemas SaaS de escala mediana.

Como prueba de concepto, el sistema cumple su objetivo pedagógico: evidenciar en código ejecutable los beneficios del desacoplamiento, la asincronía y la comunicación via mensajes en sistemas distribuidos modernos.